

```
In [ ]: # 1.2 KNN-регрессия.
```

```
In [ ]: Импорт необходимых библиотек:
```

```
In [34]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
In [ ]: ## 1. Загрузка и обработка данных.
Загружаем датасет из csv-файла. При первичной обработке выяснилось, что файл имеет десятичный разделитель ',', значения пропусков '-200'. Выводим первые 5 строк данных также для проверки корректности загрузки. Получаем общую информацию о датасете 2 столбца типа object (строковый). Значения -200 заменяем на NaN.
```

```
In [35]: df_air = pd.read_csv(
    'C:/Users/Elizaveta/Downloads/air_quality/AirQuality.csv',
    delimiter=';',
    decimal=',',
    na_values=[' ', 'NA', 'nan', -200]
)
print(f"размер после загрузки: {df_air.shape}")
print("столбцы после загрузки:", list(df_air.columns))
df_air = df_air.loc[:, ~df_air.columns.str.contains('Unnamed')]
print(f"\nразмер таблицы после удаления пустых столбцов: {df_air.shape}")
print("столбцы:", list(df_air.columns))
print(df_air.head())
print(df_air.info())
print(df_air.isnull().sum())
```

размер после загрузки: (9471, 17)

столбцы после загрузки: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH', 'Unnamed: 15', 'Unnamed: 16']

размер таблицы после удаления пустых столбцов: (9471, 15)

столбцы: ['Date', 'Time', 'CO(GT)', 'PT08.S1(CO)', 'NMHC(GT)', 'C6H6(GT)', 'PT08.S2(NMHC)', 'NOx(GT)', 'PT08.S3(NOx)', 'NO2(GT)', 'PT08.S4(NO2)', 'PT08.S5(O3)', 'T', 'RH', 'AH']

	Date	Time	CO(GT)	PT08.S1(CO)	NMHC(GT)	C6H6(GT)	\
0	10/03/2004	18.00.00	2.6	1360.0	150.0	11.9	
1	10/03/2004	19.00.00	2.0	1292.0	112.0	9.4	
2	10/03/2004	20.00.00	2.2	1402.0	88.0	9.0	
3	10/03/2004	21.00.00	2.2	1376.0	80.0	9.2	
4	10/03/2004	22.00.00	1.6	1272.0	51.0	6.5	

	PT08.S2(NMHC)	NOx(GT)	PT08.S3(NOx)	NO2(GT)	PT08.S4(NO2)	PT08.S5(O3)	\
0	1046.0	166.0	1056.0	113.0	1692.0	1268.0	
1	955.0	103.0	1174.0	92.0	1559.0	972.0	
2	939.0	131.0	1140.0	114.0	1555.0	1074.0	
3	948.0	172.0	1092.0	122.0	1584.0	1203.0	
4	836.0	131.0	1205.0	116.0	1490.0	1110.0	

	T	RH	AH
0	13.6	48.9	0.7578
1	13.3	47.7	0.7255
2	11.9	54.0	0.7502
3	11.0	60.0	0.7867
4	11.2	59.6	0.7888

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 9471 entries, 0 to 9470

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	Date	9357 non-null	object
1	Time	9357 non-null	object
2	CO(GT)	7674 non-null	float64
3	PT08.S1(CO)	8991 non-null	float64
4	NMHC(GT)	914 non-null	float64
5	C6H6(GT)	8991 non-null	float64
6	PT08.S2(NMHC)	8991 non-null	float64
7	NOx(GT)	7718 non-null	float64
8	PT08.S3(NOx)	8991 non-null	float64
9	NO2(GT)	7715 non-null	float64
10	PT08.S4(NO2)	8991 non-null	float64
11	PT08.S5(O3)	8991 non-null	float64
12	T	8991 non-null	float64
13	RH	8991 non-null	float64
14	AH	8991 non-null	float64

dtypes: float64(13), object(2)

memory usage: 1.1+ MB

None

Date 114

Time 114

CO(GT) 1797

PT08.S1(CO) 480

NMHC(GT) 8557

C6H6(GT) 480

PT08.S2(NMHC) 480

NOx(GT) 1753

```

PT08.S3(NOx)      480
NO2(GT)           1756
PT08.S4(NO2)      480
PT08.S5(O3)       480
T                 480
RH                480
AH                480
dtype: int64

```

In []: Столбцы с датой и временем удаляются за ненадобностью. В качестве целевой переменной (C6H6(GT)) выбран уровень озона. Признак 'NMHC(GT)' удаляется в виде нумерованного списка.

```

In [36]: df_air = df_air.drop(['Date', 'Time'], axis=1, errors='ignore')
target_column = 'C6H6(GT)'
column_to_drop = ['NMHC(GT)', 'Hour', 'DayOfWeek', 'Month', target_column]
numeric_columns = df_air.select_dtypes(include=[np.number]).columns
feature_columns = [columns for columns in numeric_columns if columns not in column_to_drop]
print(f"\nИспользуемые признаки:")
for i, columns in enumerate(feature_columns, 1):
    print(f"{i}. {columns}")

```

Используемые признаки:

1. CO(GT)
2. PT08.S1(CO)
3. PT08.S2(NMHC)
4. NOx(GT)
5. PT08.S3(NOx)
6. NO2(GT)
7. PT08.S4(NO2)
8. PT08.S5(O3)
9. T
10. RH
11. AH

In []: Формирование чистых данных. После определения списка признаков и целевой переменной удаляем строки с пропущенными значениями. Выводим информацию о размерах чтобы оценить потерю данных составила 27%, что является приемлемым. Статистики целевой переменной показывают её распределение. Большой разброс между средним и медианой - о симметричности распределения. Эти данные влияют на выбор

```

In [37]: X = df_air[feature_columns]
Y = df_air[target_column]
mask = X.notnull().all(axis=1)&Y.notnull()
X_clean = X[mask]
Y_clean = Y[mask]
print(f"Размер X: {X_clean.shape}")
print(f"Размер Y: {Y_clean.shape}")
print(f"\nЗначения целевой переменной {target_column}:")
print(f"Min: {Y_clean.min():.3f}")
print(f"Max: {Y_clean.max():.3f}")
print(f"Среднее: {Y_clean.mean():.3f}")
print(f"Медиана: {Y_clean.median():.3f}")
print(f"Стандартное отклонение: {Y_clean.std():.3f}")

```

Размер X: (6941, 11)

Размер Y: (6941,)

Значения целевой переменной C6H6(GT):

Min: 0.200

Max: 63.700

Среднее: 10.554

Медиана: 8.800

Стандартное отклонение: 7.465

In []: Разделение данных на признаки X и целевую переменную Y. После чего проверяем раз

```
In [38]: from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X_clean, Y_clean,
                                                    test_size=0.3,
                                                    random_state=42)

print(f"Обучающая выборка: {X_train.shape}")
print(f"Тестовая выборка: {X_test.shape}")
```

Обучающая выборка: (4858, 11)

Тестовая выборка: (2083, 11)

In []: *##2. Создание бейзлайна.*

Создаём базовую модель с использованием KNeighborsRegressor с параметрами по умолчанию. Обучающая выборка (train) используется для обучения модели.

Тестовая выборка (test) используется для оценки качества обученной модели.

Используются метрики MAE, RMSE, R2.

MAE=0.5262 показывает среднюю величину ошибки в тех же единицах, что и целевая переменная. RMSE=1.0324 также показывает среднюю ошибку переменной, но за счёт возведения в квадрат демонстрирует наличие выбросов.

R2=0.9813 показывает долю дисперсии целевой переменной, полученное значение подтверждает гипотезу. Базовая модель демонстрирует довольно высокие показатели. Она служит отправной точкой для улучшения.

```
In [39]: from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

knn_baseline = KNeighborsRegressor(n_neighbors=5)
knn_baseline.fit(X_train, Y_train)
Y_baseline_prediction = knn_baseline.predict(X_test)
MAE_baseline = mean_absolute_error(Y_test, Y_baseline_prediction)
RMSE_baseline = np.sqrt(mean_squared_error(Y_test, Y_baseline_prediction))
R2_baseline = r2_score(Y_test, Y_baseline_prediction)

print(f"\n baseline KNeighborsRegressor:")
print(f"MAE: {MAE_baseline:.4f}")
print(f"RMSE: {RMSE_baseline:.4f}")
print(f"R2: {R2_baseline:.4f}")
```

baseline KNeighborsRegressor:

MAE: 0.5262

RMSE: 1.0324

R2: 0.9813

In []: *##3. Улучшение бейзлайна.*

Гипотеза 1: Анализируем корреляцию признаков с целевой переменной.

Создаётся копия матрицы признаков, чтобы не изменять исходные данные, к ней добавляется целевая переменная. Вычисляем корреляционную матрицу, она визуализирована в виде heatmap. Корреляция превышает 0.7. В результате определена очень высокая корреляция (0.98), т.к. практически линейно определяет целевую переменную.

```
In [40]: import seaborn as sns
import matplotlib.pyplot as plt
```

```

df_corr = X_clean.copy()
df_corr[target_column] = Y_clean
corr_matrix = df_corr.corr()
print("Корреляции с целевой переменной C6H6:")
corr_with_target = corr_matrix[target_column].sort_values(ascending=False)
print(corr_with_target)
plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, annot=True,fmt='.2f', cmap='coolwarm', center=0, square)
plt.tight_layout()
plt.show()
print(f"\nПризнаки с высокой корреляцией:")
for feature, corr_value in corr_with_target.items():
    if feature !=target_column and abs(corr_value)>0.7:
        print(f"{feature}: {corr_value:.3f}")

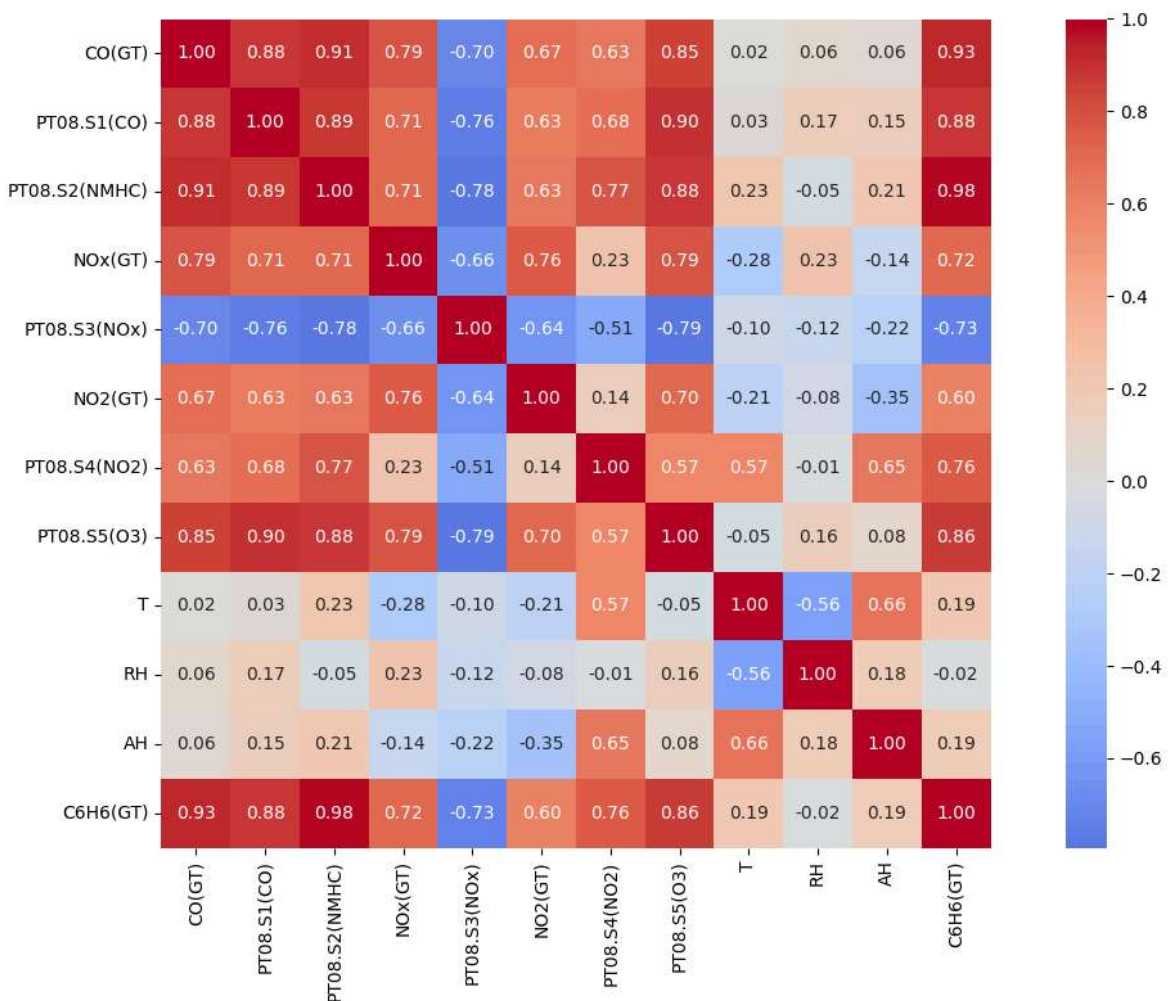
```

Корреляции с целевой переменной C6H6:

```

C6H6(GT)      1.000000
PT08.S2(NMHC) 0.982705
CO(GT)        0.930008
PT08.S1(CO)   0.877430
PT08.S5(O3)   0.861154
PT08.S4(NO2)  0.761805
NOx(GT)       0.718344
NO2(GT)       0.603241
T             0.189003
AH            0.187072
RH           -0.021592
PT08.S3(NOx) -0.725722
Name: C6H6(GT), dtype: float64

```



Признаки с высокой корреляцией:

PT08.S2(NMHC): 0.983

CO(GT): 0.930

PT08.S1(CO): 0.877

PT08.S5(O3): 0.861

PT08.S4(NO2): 0.762

NOx(GT): 0.718

PT08.S3(NOx): -0.726

In []: Оценим, насколько изменилась модель после удаления сильно коррелированного признака с 0.9813 до 0.9417 (разница 0.0396). Этот признак действительно сильно повышал остальные признаки обладают достаточной предсказательной способностью. Рост MAE удаления сильно коррелированного признака будут получены более объективные данные

```
In [41]: new_feature_columns = [column for column in feature_columns if column != 'PT08.S2(
X_new = X_clean[new_feature_columns]
X_train_new, X_test_new, Y_train_new, Y_test_new = train_test_split(X_new, Y_clean,
knn_new = KNeighborsRegressor(n_neighbors=5)
knn_new.fit(X_train_new, Y_train_new)
Y_new_prediction = knn_new.predict(X_test_new)
MAE_new = mean_absolute_error(Y_test_new, Y_new_prediction)
RMSE_new = np.sqrt(mean_squared_error(Y_test_new, Y_new_prediction))
R2_new = r2_score(Y_test_new, Y_new_prediction)
print(f"\n baseline KNeighborsRegressor:")
print(f"MAE: {MAE_new:.4f}")
print(f"RMSE: {RMSE_new:.4f}")
print(f"R2: {R2_new:.4f}")
print(f" Разница в R2: {abs(R2_baseline-R2_new):.4f}")
```

baseline KNeighborsRegressor:

MAE: 1.1282

RMSE: 1.8236

R2: 0.9417

Разница в R2: 0.0396

In [16]: *## 4. Имплементация алгоритма машинного обучения.*
Представлена самостоятельная реализация алгоритма KNN для задачи регрессии.
__init__ - задание гиперпараметров.
fit - сохранение обучающих данных.
_compute_distance - вычисление расстояний.
predict - формирование предсказания.

Cell In[16], line 2

Представлена самостоятельная реализация алгоритма KNN для задачи регрессии.

^

SyntaxError: invalid syntax

```
In [42]: class MyKNNRegressor:
def __init__(self, n_neighbors=5, weights='uniform', metric='euclidean'):
self.n_neighbors = n_neighbors
self.weights = weights
self.metric = metric
self.X_train = None
self.Y_train = None
def fit(self, X, Y):
self.X_train = np.array(X)
self.Y_train = np.array(Y)
return self
def _compute_distance(self, point1, point2):
if self.metric == 'euclidean':
```

```

        return np.sqrt(np.sum((point1-point2)**2))
    elif self.metric == 'manhattan':
        return np.sum(np.abs(point1-point2))
    else:
        raise ValueError(f"Метрика {self.metric} не поддерживается")
def predict(self, X):
    X = np.array(X)
    predictions = []
    for point in X:
        distances = []
        for i, train_point in enumerate(self.X_train):
            distance = self._compute_distance(point, train_point)
            distances.append((i, distance))
        distances.sort(key=lambda x: x[1])
        neighbors = distances[:self.n_neighbors]
        if self.weights == 'uniform':
            neighbor_values = [self.Y_train[idx] for idx, _ in neighbors]
            predictions.append(np.mean(neighbor_values))
        elif self.weights == 'distance':
            weighted_sum = 0
            total_weight = 0
            for idx, distance in neighbors:
                weight = 1.0/(distance+1e-8)
                weighted_sum +=weight*self.Y_train[idx]
                total_weight +=weight
            predictions.append(weighted_sum/total_weight)
    return np.array(predictions)

```

In []: На собственной модели получены результаты, идентичные результатам бейзлайна, что
 RMSE: 1.8236
 R2: 0.9417

```

In [43]: my_knn = MyKNNRegressor(n_neighbors=5, weights='uniform', metric='euclidean')
my_knn.fit(X_train_new, Y_train_new)
Y_my_knn_prediction = my_knn.predict(X_test_new)
MAE_my_knn = mean_absolute_error(Y_test_new, Y_my_knn_prediction)
RMSE_my_knn = np.sqrt(mean_squared_error(Y_test_new, Y_my_knn_prediction))
R2_my_knn = r2_score(Y_test_new, Y_my_knn_prediction)
print(f"\nMyKNNRegressor:")
print(f"MAE: {MAE_my_knn:.4f}")
print(f"RMSE: {RMSE_my_knn:.4f}")
print(f"R2: {R2_my_knn:.4f}")

```

MyKNNRegressor:
 MAE: 1.1282
 RMSE: 1.8236
 R2: 0.9417

In []: *## Гипотеза 2: Масштабирование признаков.*
 Признаки, имеющие разные единицы измерения или порядки величин, могут неравномер
 и единичной дисперсии устраняет этот эффект и повышает качество модели. После ма
 собственную реализацию. Метрики показывают, что масштабирование улучшило качеств
 Sklearn: R2=0.9592, MAE=0.8576, RMSE=1.5262
 MyKNNRegressor: R2=0.9592, MAE=0.8576, RMSE=1.5262

```

In [44]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_new)

```



```

X_test_scaled = scaler.transform(X_test_new)
knn_scaled_sklearn = KNeighborsRegressor(n_neighbors=5)
knn_scaled_sklearn.fit(X_train_scaled, Y_train_new)
Y_prediction_scaled_sklearn = knn_scaled_sklearn.predict(X_test_scaled)
R2_scaled_sklearn = r2_score(Y_test_new, Y_prediction_scaled_sklearn)
MAE_scaled_sklearn = mean_absolute_error(Y_test_new, Y_prediction_scaled_sklearn)
RMSE_scaled_sklearn = np.sqrt(mean_squared_error(Y_test_new, Y_prediction_scaled

my_knn_scaled = MyKNNRegressor(n_neighbors=5)
my_knn_scaled.fit(X_train_scaled, Y_train_new)
Y_prediction_my_knn_scaled = my_knn_scaled.predict(X_test_scaled)
R2_my_knn_scaled = r2_score(Y_test_new, Y_prediction_my_knn_scaled)
MAE_my_knn_scaled = mean_absolute_error(Y_test_new, Y_prediction_my_knn_scaled)
RMSE_my_knn_scaled = np.sqrt(mean_squared_error(Y_test_new, Y_prediction_my_knn_

print(f"\nУлучшения после масштабирования:")
print(f"Sklearn: R2={R2_scaled_sklearn:.4f}, MAE={MAE_scaled_sklearn:.4f}, RMSE=
print(f"MyKNNRegressor: R2={R2_my_knn_scaled:.4f}, MAE={MAE_my_knn_scaled:.4f},

```

Улучшения после масштабирования:

Sklearn: R2=0.9592, MAE=0.8576, RMSE=1.5262

MyKNNRegressor: R2=0.9592, MAE=0.8576, RMSE=1.5262

In []: *## Гипотеза 3: Подбор гиперпараметров.*

Подбор наилучшей комбинации параметров, обеспечивающей максимальное качество пре-
расстояния. Используется 5-кратная кросс-валидация. Для каждой комбинации вычисл
комбинация запоминается. Оптимальное число соседей оказалось равным 7, манхэттенс
преимущество перед равномерным усреднением. Лучший R2 на кросс-валидации: 0.9687
R2 на тесте: 0.9622186232746189

```

In [45]: from sklearn.model_selection import GridSearchCV
param_grid = {'n_neighbors': range(1,31), 'weights':['uniform', 'distance'], 'me
grid_search = GridSearchCV(KNeighborsRegressor(), param_grid, cv=5, scoring='r2'
grid_search.fit(X_train_scaled, Y_train_new)
print("Лучшие параметры:", grid_search.best_params_)
print("Лучший R2 на кросс-валидации:", grid_search.best_score_)
print("R2 на тесте:", grid_search.score(X_test_scaled, Y_test_new))

```

Fitting 5 folds for each of 120 candidates, totalling 600 fits

Лучшие параметры: {'metric': 'manhattan', 'n_neighbors': 7, 'weights': 'distanc
e'}

Лучший R2 на кросс-валидации: 0.9687728169534437

R2 на тесте: 0.9622186232746189

In []: Собственная реализация с оптимальными параметрами. обучаем собственную модель на
Полученные метрики показывают улучшение качества модели и корректность алгоритм
MAE: 0.8195
RMSE: 1.4678
R2: 0.9622

```

In [46]: my_knn_best = MyKNNRegressor(n_neighbors=grid_search.best_params_['n_neighbors'])
my_knn_best.fit(X_train_scaled, Y_train_new)
Y_prediction_my_knn_best = my_knn_best.predict(X_test_scaled)
MAE_my_knn_best = mean_absolute_error(Y_test_new, Y_prediction_my_knn_best)
RMSE_my_knn_best = np.sqrt(mean_squared_error(Y_test_new, Y_prediction_my_knn_be
R2_my_knn_best = r2_score(Y_test_new, Y_prediction_my_knn_best)
print("\n Результаты MyKNNRegressor на лучших параметрах:")
print(f"MAE: {MAE_my_knn_best:.4f}")
print(f"RMSE: {RMSE_my_knn_best:.4f}")
print(f"R2: {R2_my_knn_best:.4f}")

```


Результаты MyKNNRegressor на лучших параметрах:
MAE: 0.8195
RMSE: 1.4678
R2: 0.9622

```
In [ ]: ## Выводы:
```

В ходе выполнения задания была выполнена предобработка данных и удаления признаков реалистичных данных и повысить устойчивость модели. Была создана базовая модель, MAE: 1.1282, RMSE: 1.8236, R2: 0.9417. После масштабирования признаков качество модели выросло. прирост R2 составил 1.8 улучшения, R2 на тесте: 0.9622. Была создана собственная реализация алгоритма My библиотечной модели, что подтверждает корректность реализации. После масштабирования MAE: 0.8195, RMSE: 1.4678, R2: 0.9622, что говорит о влиянии этих улучшений на качество модели. Таким образом, каждый этап улучшения вносил вклад в итоговое качество, а собственная